

✓ TodoApp

Full-Stack Django & Bootstrap Web Application

Document Type	Technical Documentation & Code Explanation
Project Name	TodoApp – Django & Bootstrap Todo List
Framework	Django 4.x + Bootstrap 5
Architecture	Function-Based Views (FBV)
Database	SQLite (Development)
Audience	Computer Science Students

This document is a complete guide to the TodoApp project. It explains every file, every code decision, and every Django concept used — written specifically for students learning web development with Python and Django.

Table of Contents

1. Project Overview & Features
2. Project Structure & File Organization
3. How Django Works – Key Concepts
4. Database Models (models.py)
5. Forms (forms.py)
6. Views (views.py)
7. URL Configuration (urls.py)
8. Templates & HTML
9. Authentication System
10. Running the Application
11. Key Django Concepts Explained
12. Bootstrap 5 Integration

1. Project Overview & Features

ToDoApp is a fully-functional web application that allows users to manage their daily tasks (todos). It is built with Django, a powerful Python web framework, and styled using Bootstrap 5, a popular CSS framework.

Features Implemented:

Feature	Description
User Authentication	Register, login, logout with Django's built-in auth system
Dashboard	Overview of all tasks with statistics cards and a progress bar
Create Tasks	Add tasks with title, description, priority, category, and due date
Edit Tasks	Update any existing task via a dedicated edit page
Delete Tasks	Permanently delete tasks with a confirmation modal
Toggle Complete	Mark tasks as done/undone with a single click
Filter & Search	Filter by status, priority, and category; search by keyword
Categories	Create color-coded categories to organize tasks
User Profile	View and edit profile info including bio and avatar color
Change Password	Securely update password while staying logged in
Overdue Detection	Automatically flags tasks past their due date
Admin Panel	Django's built-in admin for database management

2. Project Structure & File Organization

Every Django project follows a specific folder structure. Understanding this structure is essential before reading any code. Here is how the files are organized:

```
todoproject/          ← Root project folder
├── manage.py          ← Django's command-line tool (run server, migrate, etc.)
├── db.sqlite3          ← SQLite database file (auto-created after migration)
├── documentation.pdf  ← This document
├── todoproject/       ← Project configuration package
│   ├── __init__.py    ← Marks this as a Python package
│   ├── settings.py    ← All project settings (database, apps, templates...)
│   ├── urls.py        ← Root URL configuration
│   ├── wsgi.py        ← Web server entry point for deployment
│   └── todos/         ← Our main app (all business logic lives here)
│       ├── __init__.py
│       ├── admin.py   ← Registers models with Django Admin
│       ├── apps.py    ← App configuration
│       ├── forms.py   ← Form classes for user input
│       ├── models.py  ← Database models (the data structure)
│       ├── views.py   ← All view functions (request → response)
│       ├── migrations/
│       │   └── 0001_initial.py ← First migration (creates tables)
│       └── templates/ ← All HTML template files
│           ├── base.html ← Master layout (header, sidebar, footer)
│           ├── home.html ← Public landing page
│           ├── login.html ← Login form page
│           ├── register.html ← Registration form page
│           ├── dashboard.html ← Main todo dashboard
│           ├── edit_todo.html ← Edit a specific task
│           ├── profile.html ← User profile page
│           ├── edit_profile.html ← Edit profile form
│           └── change_password.html ← Change password form
├── static/
│   └── css/
│       └── style.css ← Custom CSS styles
```

IMPORTANT: Django separates your project into 'apps'. An app is a module that handles one specific feature. Our app is called 'todos' and it handles everything related to tasks.

3. How Django Works – Key Concepts

3.1 The Request-Response Cycle

When a user visits a URL in their browser, here is exactly what happens in Django:

Step	What Happens	Django Component
1	User types URL in browser (e.g. /dashboard/)	Browser
2	Django checks urls.py to find a matching URL pattern	urls.py
3	Django calls the matching view function	views.py
4	The view queries the database via models	models.py
5	The view passes data to an HTML template	templates/
6	Django renders the template and returns HTML	Template Engine
7	Browser displays the HTML page to the user	Browser

3.2 MTV Architecture

Django uses the MTV pattern: Model, Template, View. This is similar to the well-known MVC pattern but with slightly different naming:

Layer	Django Name	Responsibility
Data	Model (models.py)	Defines the database structure and handles data logic
Display	Template (.html files)	Defines what the user sees (HTML + dynamic data)
Logic	View (views.py)	Connects models and templates; handles business logic

4. Database Models (models.py)

Models are Python classes that define your database tables. Each class = one table. Each attribute = one column. Django automatically creates the SQL for you.

4.1 The Category Model

Categories group related tasks together. Each category has a name, a hex color, and is linked to one user (ForeignKey).

```
class Category(models.Model):
    name = models.CharField(max_length=100)          # Short text field (max 100 chars)
    color = models.CharField(max_length=7, default='#0d6efd') # Hex color like #ff0000
    user = models.ForeignKey(                        # Many-to-one: many categories per user
        User,
        on_delete=models.CASCADE,                    # Delete categories when user is deleted
        related_name='categories'                    # Allows user.categories.all()
    )
    created_at = models.DateTimeField(auto_now_add=True) # Set once on creation

    class Meta:
        ordering = ['name']                          # Always sort alphabetically by name
```

4.2 The Todo Model

The main model. Each todo has a title, optional description, priority level, completion status, and optional due date.

```
class Todo(models.Model):
    PRIORITY_CHOICES = [                             # Choices restrict values to only these options
        ('low', 'Low'),
        ('medium', 'Medium'),
        ('high', 'High'),
    ]

    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='todos')
    category = models.ForeignKey(
        Category,
        on_delete=models.SET_NULL,                    # If category deleted, set this field to NULL
        null=True, blank=True                          # This field is optional
    )
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True, null=True) # Longer text, optional
    priority = models.CharField(max_length=10, choices=PRIORITY_CHOICES, default='medium')
    is_completed = models.BooleanField(default=False)    # True/False field
    due_date = models.DateField(null=True, blank=True)  # Date only (no time)
    created_at = models.DateTimeField(auto_now_add=True) # Auto-set on create
    updated_at = models.DateTimeField(auto_now=True)    # Auto-update on every save

    def is_overdue(self):
        from django.utils import timezone
        if self.due_date and not self.is_completed:
            return self.due_date < timezone.now().date()
        return False
```

The `is_overdue()` method is a custom method we added to the model. It checks if a task's due date has passed and the task is still incomplete. You can call it in templates like: `{{ todo.is_overdue }}`

4.3 The UserProfile Model

Django's built-in User model doesn't have fields like bio or avatar color. We extend it using a `OneToOneField` — each user has exactly one profile.

```
class UserProfile(models.Model):
    # OneToOneField means: one user ↔ one profile (unlike ForeignKey which allows many)
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
    bio = models.TextField(blank=True, null=True)
    avatar_color = models.CharField(max_length=7, default='#0d6efd')
```

5. Forms (forms.py)

Forms handle user input safely. Django forms validate data, display error messages, and protect against security attacks like CSRF. We use two types of forms:

Form Type	When to Use	Example in our app
forms.Form	For custom forms not tied to a model	LoginForm
forms.ModelForm	For forms that directly save to a model	TodoForm, CategoryForm

5.1 RegisterForm

```
class RegisterForm(UserCreationForm):
    # We extend Django's built-in UserCreationForm to add extra fields
    email = forms.EmailField(required=True, widget=forms.EmailInput(
        attrs={'class': 'form-control', 'placeholder': 'Enter your email'}
    ))

    class Meta:
        model = User
        fields = ['username', 'first_name', 'last_name', 'email', 'password1', 'password2']

    def save(self, commit=True):
        user = super().save(commit=False) # Don't save to DB yet
        user.email = self.cleaned_data['email'] # Manually add email
        if commit:
            user.save() # Now save to DB
        return user
```

5.2 TodoForm – The `__init__` Override

This is a key concept: we override `__init__` to filter the category dropdown so users only see their own categories, not everyone else's.

```
class TodoForm(forms.ModelForm):
    class Meta:
        model = Todo
        fields = ['title', 'description', 'priority', 'category', 'due_date']

    def __init__(self, user, *args, **kwargs):
        # We pass the logged-in user as the first argument
        super().__init__(*args, **kwargs) # Call parent __init__ normally
        # Filter categories to only show THIS user's categories
        self.fields['category'].queryset = Category.objects.filter(user=user)
        self.fields['category'].required = False # Make it optional
```

6. Views (views.py)

Views are Python functions that receive an HTTP request and return an HTTP response. They contain all the business logic of our application.

6.1 The @login_required Decorator

A decorator modifies a function's behavior. @login_required checks if the user is logged in before running the view. If not, it redirects to the login page.

```
@login_required          # ← This decorator protects the view
def dashboard(request):   # ← Only runs if user is logged in
    ...

# Without @login_required, anyone could visit /dashboard/ without logging in!
# Django handles the redirect automatically to the LOGIN_URL in settings.py
```

6.2 GET vs POST Requests

Every view checks the request method. GET = user is viewing a page. POST = user submitted a form.

```
def login_view(request):
    if request.method == 'POST':          # Form was submitted
        form = LoginForm(request.POST)    # Bind the submitted data
        if form.is_valid():
            # Process the login...
    else:                                  # GET request – user just visited the page
        form = LoginForm()                 # Empty form
    return render(request, 'login.html', {'form': form})
```

6.3 The Dashboard View – Filtering

The dashboard demonstrates how to filter a queryset based on URL parameters (like ?status=active or ?priority=high).

```
@login_required
def dashboard(request):
    # Start with ALL todos belonging to this user
    todos = Todo.objects.filter(user=request.user)

    # Read filter values from the URL query string (?status=active&priority=high)
    filter_status = request.GET.get('status', 'all') # Default to 'all'

    # Apply filters conditionally
    if filter_status == 'active':
        todos = todos.filter(is_completed=False)
    elif filter_status == 'overdue':
        todos = todos.filter(is_completed=False, due_date__lt=timezone.now().date())

    # Pass filtered todos to the template
    return render(request, 'dashboard.html', {'todos': todos})
```

6.4 get_object_or_404 – Safety Pattern

When editing or deleting a specific todo, we use `get_object_or_404`. This is a security best practice.

```
@login_required
def edit_todo(request, pk):
    # This does THREE things at once:
    # 1. Finds the todo with this primary key (pk)
    # 2. Ensures it belongs to the logged-in user (user=request.user)
    # 3. Returns a 404 error if not found (instead of crashing)
    todo = get_object_or_404(Todo, pk=pk, user=request.user)
    # Without user=request.user, user A could edit user B's todos!
```

7. URL Configuration (urls.py)

urls.py maps URL patterns to view functions. Think of it as a table of contents that tells Django which view to call for each URL.

```
from django.urls import path
from todos import views

urlpatterns = [
    # Public pages (no login needed)
    path('', views.home, name='home'),          # /
    path('register/', views.register_view, name='register'), # /register/
    path('login/', views.login_view, name='login'), # /login/

    # Protected pages (login required)
    path('dashboard/', views.dashboard, name='dashboard'),

    # URL with a parameter: <int:pk> captures a number from the URL
    # e.g. /todo/5/edit/ → calls edit_todo(request, pk=5)
    path('todo/<int:pk>/edit/', views.edit_todo, name='edit_todo'),
    path('todo/<int:pk>/delete/', views.delete_todo, name='delete_todo'),
    path('todo/<int:pk>/toggle/', views.toggle_todo, name='toggle_todo'),
]

# In templates, use {% url 'name' %} to generate URLs:
# {% url 'edit_todo' todo.pk %} generates /todo/5/edit/
```

The name= parameter in each path() is crucial. It lets you reference URLs by name in templates and views instead of hardcoding the URL. If you change a URL later, you only need to update urls.py — not every template.

8. Templates & HTML

8.1 Template Inheritance – base.html

Instead of repeating the same HTML in every file, we use template inheritance. `base.html` contains the common layout (sidebar, navbar, scripts). Every other page extends it and only fills in the unique parts.

```
<!-- base.html defines blocks that child templates can fill -->
<html>
  <head>{% block title %}TodoApp{% endblock %}</head>
  <body>
    <nav>...sidebar...</nav>
    <main>
      {% block content %}<!-- Child templates put their content here -->{% endblock %}
    </main>
  </body>
</html>

<!-- dashboard.html extends base and fills in the block -->
{% extends 'base.html' %}
{% block title %}Dashboard - TodoApp{% endblock %}
{% block content %}
  <h1>My Tasks</h1>
  ...dashboard content...
{% endblock %}
```

8.2 Template Tags & Filters

```
<!-- Variables: double curly braces display values -->
{{ user.username }}
{{ todo.title }}

<!-- Tags: curly brace with percent sign for logic -->
{% if todo.is_completed %}           <!-- Conditional -->
  <s>Done!</s>
{% else %}
  <span>Not done</span>
{% endif %}

{% for todo in todos %}              <!-- Loop -->
  <p>{{ todo.title }}</p>
{% empty %}                          <!-- Shown when list is empty -->
  <p>No tasks yet!</p>
{% endfor %}

<!-- Filters: transform a value using | -->
{{ todo.due_date|date:"M j, Y" }}    <!-- Format date as "May 16, 2026" -->
{{ todo.title|upper }}               <!-- Uppercase: "MY TASK" -->
{{ todo.description|truncatewords:20 }} <!-- Show first 20 words -->

<!-- URL tag generates a link from a named URL -->
<a href="{% url 'edit_todo' todo.pk %}">Edit</a>

<!-- CSRF token - required in every POST form for security -->
```


9. Authentication System

Django comes with a complete authentication system built-in. We use it rather than building our own, which would be insecure.

9.1 Key Auth Functions

```
from django.contrib.auth import login, logout, authenticate

# authenticate() checks credentials against the database
# Returns a User object if valid, None if invalid
user = authenticate(request, username='alice', password='secret123')

# login() creates a session (remembers the user between requests)
if user is not None:
    login(request, user)

# logout() destroys the session
logout(request)

# update_session_auth_hash() keeps user logged in after password change
# (Django invalidates sessions when password changes for security)
update_session_auth_hash(request, user)
```

9.2 CSRF Protection

CSRF (Cross-Site Request Forgery) is a security attack where a malicious website tricks a logged-in user into submitting a form they didn't intend to. Django prevents this with a secret token in every form.

```
<!-- Add this inside EVERY <form method='post'> tag -->
<form method="post">
    {% csrf_token %}
    <!-- Django generates: <input type="hidden" name="csrfmiddlewaretoken" value="abc123..."> -->
    <!-- If the token is missing or wrong, Django rejects the request -->
</form>
```

9.3 Settings Related to Auth

```
# settings.py
LOGIN_URL = '/login/' # Where to redirect if @login_required fails
LOGIN_REDIRECT_URL = '/dashboard/' # Where to go after successful login
LOGOUT_REDIRECT_URL = '/login/' # Where to go after logout
```

10. Running the Application

Step-by-Step Setup Instructions

Step 1: Install Python & Django

```
$ pip install django
```

Make sure Python 3.8+ is installed first.

Step 2: Navigate to the project folder

```
$ cd todoproject
```

Use your terminal / command prompt.

Step 3: Run database migrations

```
$ python manage.py migrate
```

Creates all database tables from models.py.

Step 4: Create an admin user (optional)

```
$ python manage.py createsuperuser
```

Follow the prompts. Access admin at /admin/

Step 5: Start the development server

```
$ python manage.py runserver
```

The server starts at http://127.0.0.1:8000/

Step 6: Open your browser

```
$ http://127.0.0.1:8000/
```

Register a new account and start adding tasks!

Common Management Commands

Command	What it Does
<code>python manage.py runserver</code>	Start development server on port 8000
<code>python manage.py migrate</code>	Apply all pending database migrations
<code>python manage.py makemigrations</code>	Create migration files after changing models.py
<code>python manage.py createsuperuser</code>	Create an admin account
<code>python manage.py shell</code>	Open an interactive Python shell with Django loaded
<code>python manage.py check</code>	Check for configuration errors

11. Key Django Concepts Explained

11.1 ORM & QuerySets

Django's ORM lets you interact with the database using Python instead of SQL. QuerySets are lazy and only hit the database when used.

```
# SQL version (what happens under the hood):
# SELECT * FROM todos_todo WHERE user_id = 1 AND is_completed = 0

# Django ORM version (what you write):
todos = Todo.objects.filter(user=request.user, is_completed=False)

# More ORM examples:
todo = Todo.objects.get(pk=5)           # Get one specific todo
Todo.objects.all()                      # Get all todos
Todo.objects.count()                    # Count todos
Todo.objects.order_by('-created_at')    # Sort newest first
Todo.objects.filter(priority='high')    # Filter
```

11.2 Signals (Auto-Creating Profiles)

In our app, we create the UserProfile inside register_view manually. An alternative is Django Signals — automatic triggers when model events happen. For reference, here's how you'd do it with signals:

```
from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    # This runs automatically every time a User is saved
    if created: # Only on first creation, not updates
        UserProfile.objects.create(user=instance)
```

11.3 Context Processors

Context processors automatically add variables to every template. That's why `{{ user }}` and `{{ request }}` work in every template without explicitly passing them from the view.

12. Bootstrap 5 Integration

Bootstrap 5 is loaded from a CDN (Content Delivery Network) — no installation needed. We use it for responsive layout, components, and utilities.

Bootstrap Feature	How We Use It	Example Class
Grid System	Responsive columns that adapt to screen size	col-md-4, col-12
Cards	Rounded boxes for stat cards and todo items	card, card-body
Buttons	Styled action buttons	btn btn-primary, btn btn-danger
Modals	Popup dialogs for adding tasks	modal, modal-dialog
Alerts	Flash messages for feedback	alert alert-success
Forms	Styled inputs and selects	form-control, form-select
Badges	Small colored labels for priority	badge bg-danger
Progress	Completion progress bar	progress, progress-bar
Utilities	Spacing, colors, display	mb-3, text-muted, d-flex

12.1 Modal Example

```
<!-- Button that triggers the modal -->
<button data-bs-toggle="modal" data-bs-target="#addTodoModal">
  Add Task
</button>

<!-- The modal itself (hidden by default, shown when button clicked) -->
<div class="modal fade" id="addTodoModal">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Add New Task</h5>
        <button class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <!-- Form goes here -->
      </div>
    </div>
  </div>
</div>
```

Congratulations! You now have a complete understanding of how this Django application works — from the database models all the way to the HTML templates. This project covers the core skills needed for Django web development.

Happy coding! Start by running the server, register an account, and explore the code while using the app. The best way to learn is to read the code and experiment.